EK-04 Technical Competency and Quality Standards: TEC-001 to TEC-006, QUA-001 to QUA-003

Document ID: EK-04 | Category C: Evaluation knowledge base

1. Document metadata

Field	Value
Document ID	EK-04
Filename	EK-04_Technical_Competency_and_Quality_Standards.pdf
Category	C - Evaluation knowledge base
Version	1.0
Creation date	2026-10-14
Data status	Factual technical guidance (general engineering concepts) with synthetic example references | Factual guidance on quality indicators with synthetic (fictional) examples clearly labelled
Main topics	Frontend engineering; Angular; TypeScript; State management; Accessibility; Frontend testing; Backend engineering; Spring Boot; REST API design; Authentication; Integration testing; Database design; PostgreSQL; Indexing
Relevant skills and technologies	Angular; TypeScript; RxJS; NgRx; Unit testing; Accessibility; Spring Boot; Spring Security; REST API design; Redis; Integration testing; PostgreSQL; Flyway migrations; Python; scikit-learn; PyTorch; FastAPI; Vector search; RAG pipeline; LLM evaluation; Docker; Kubernetes; CI/CD; Terraform; AWS; Monitoring; Flutter; Dart; Offline storage; Firebase push
Relevant entity IDs	TEC-001, TEC-002, TEC-003, TEC-004, TEC-005, TEC-006, QUA-001, QUA-002, QUA-003, FRE-001, FRE-002, PRJ-001, PRJ-002, FRE-003, PRJ-004, PRJ-003, FRE-004, FRE-005, PRJ-005, FRE-006, PRJ-006

Purpose and scope: this PDF consolidates 9 source documents (TEC-001 to QUA-003). Each source keeps its own ID in every section heading, so chunks stay attributable, for example [DOC:TEC-001/...].

2. Executive summary

Frontend competency means building maintainable, accessible, tested user interfaces, not merely listing a framework. This guide defines Angular, TypeScript, RxJS and NgRx competency levels and explains why a CV line such as 'Angular developer' does not prove advanced proficiency. Backend competency covers API design, data access, security, error handling, testing and operability. This guide defines levels with Spring Boot examples and states what evidence separates practical experience from a keyword on a CV. This PDF contains 9 source documents in total.

3. TEC-001: TEC-001 Frontend Engineering Competency Guide

Source document ID: TEC-001 | Data status: Factual technical guidance (general engineering concepts) with synthetic example references

Purpose and scope: define what frontend competency means for Angular-based web applications and how to judge evidence of it when comparing freelancer proposals. It covers technologies, levels, assessment questions and evidence rules.

TEC-001 executive summary

Frontend competency means building maintainable, accessible, tested user interfaces, not merely listing a framework. This guide defines Angular, TypeScript, RxJS and NgRx competency levels and explains why a CV line such as 'Angular developer' does not prove advanced proficiency.

TEC-001 - Technology and competency definitions

Angular is a TypeScript-based framework for single-page applications (SPA) with components, dependency injection, routing and a template compiler. TypeScript (TS) adds static types to JavaScript. RxJS (Reactive Extensions for JavaScript) models asynchronous data as streams called observables. NgRx is a state-management library for Angular based on the Redux pattern (actions, reducers, selectors, effects).

Frontend competency covers: component design, state management, routing and lazy loading, forms and validation, HTTP integration, performance, accessibility (a11y, meeting WCAG guidelines), and automated testing.

Example: in the fictional PRJ-001, FRE-002 demonstrated Angular competency by implementing the product filtering component, pagination and its unit tests.

TEC-001 - Beginner, intermediate and advanced competency indicators

Level	Observable indicators
Beginner (L1-L2)	Builds components from a specification; uses HttpClient for simple calls; limited tests; copies patterns without explaining trade-offs.
Intermediate (L3)	Designs reusable components; uses reactive forms; handles errors and loading states; writes component and service tests; uses RxJS operators such as switchMap and debounceTime correctly.
Advanced (L4)	Chooses state-management boundaries (local state vs NgRx); applies lazy loading and change-detection tuning; audits accessibility; sets up lint and test standards for a team.

TEC-001 - Practical implementation examples

Search-as-you-type: a filter input emits values; debounceTime(300) waits for a pause; distinctUntilChanged removes repeats; switchMap cancels the previous request. This avoids race conditions where an older response overwrites a newer one (used in PRJ-001 by FRE-002).

Dashboard state: NgRx effects call the proposals API and selectors expose derived data such as 'proposals sorted by score'. Components stay presentational (used in PRJ-002 by FRE-002).

TEC-001 - Architecture and design considerations

Separate smart (container) components from presentational components. Keep API access in services. Use typed interfaces shared with the API contract (OpenAPI). Use lazy-loaded feature modules or standalone routes to reduce initial bundle size. Avoid global mutable state outside the store.

TEC-001 - Testing and maintainability expectations

Expected: unit tests for components and services (Jasmine/Karma or Jest), selector tests for NgRx, and at least smoke-level end-to-end tests for critical flows. Maintainability indicators: strict TypeScript, linting, consistent folder structure, and no business rules duplicated in templates.

TEC-001 - Common technical mistakes

- Subscribing to observables manually without unsubscribing, causing memory leaks.
- Nested subscriptions instead of operators such as switchMap.
- Putting all state in NgRx even when local component state is enough.
- Ignoring accessibility (missing labels, no keyboard navigation).
- Disabled strict mode and widespread use of the any type.

TEC-001 - Security, scalability and reliability concerns

Security: Angular escapes template bindings by default; bypassing sanitization (bypassSecurityTrust*) is a risk. Tokens should not be stored where cross-site scripting could read them without understanding the trade-off. Scalability: bundle budgets and lazy loading. Reliability: error states, retry rules and optimistic update rollback.

TEC-001 - Practical questions and assessment scenarios

Scenario or question	What a strong answer shows
A list filter fires many requests and old results appear. How do you fix it?	Names debounce, switchMap cancellation and loading/error states; mentions tests.
When is NgRx justified?	Shared state across many routes, complex async flows, need for traceability; also admits when it is excessive.
How do you verify keyboard accessibility?	Tab order, focus management, ARIA labels, screen-reader test, automated audit plus manual checks.

TEC-001 - Evidence that demonstrates competency

- Repository code showing component plus tests for the claimed feature.
- Pull requests or commit history attributable to the freelancer ID.
- Test reports or coverage figures.
- Bundle-size or Lighthouse report for performance claims.
- Accessibility audit notes.

TEC-001 - Distinguishing superficial familiarity from practical experience

Superficial familiarity: lists Angular, cannot explain change detection or why switchMap differs from mergeMap, no tests. Practical experience: names a delivered feature, the bug encountered, the fix and the test that guards it.

A CV listing Angular does not prove advanced proficiency because the CV does not show scope of contribution, quality or independence. The dataset treats such a line as an unverified claim until project evidence exists (see EVA-001).

TEC-001 - Related skills and alternative terminology

Related skills: TypeScript, RxJS, NgRx, HTML/CSS, Angular Material, accessibility, Jasmine, Karma, Jest, Cypress, Playwright, SPA, component-based UI, state management.

TEC-001 - Summary of key findings

- Frontend competency is shown by features, tests and decisions, not by technology lists.
- RxJS operator choice and state-management boundaries separate L3 from L4.
- Missing tests or accessibility evidence should be reported as missing evidence.

TEC-001 - Keywords and alternative terminology

Angular; frontend engineering; SPA; TypeScript; RxJS; NgRx; state management; Jasmine; Karma; accessibility; WCAG; lazy loading; front end; UI developer; web UI

TEC-001 - References

- Angular documentation (angular.dev), RxJS documentation (rxjs.dev), NgRx documentation (ngrx.io), WCAG overview (w3.org/WAI). Sources named for orientation; links were not fetched during generation.

4. TEC-002: TEC-002 Backend and API Engineering Competency Guide

Source document ID: TEC-002 | Data status: Factual technical guidance (general engineering concepts) with synthetic example references

Purpose and scope: define backend and API competency, centred on Spring Boot and REST APIs, and explain evidence expected when a freelancer claims backend skills.

TEC-002 executive summary

Backend competency covers API design, data access, security, error handling, testing and operability. This guide defines levels with Spring Boot examples and states what evidence separates practical experience from a keyword on a CV.

TEC-002 - Technology and competency definitions

Spring Boot is a Java framework that simplifies building production-ready Spring applications with auto-configuration. A REST (Representational State Transfer) API exposes resources over HTTP with consistent methods, status codes and error contracts. OpenAPI is a specification format for documenting HTTP APIs. JWT (JSON Web Token) is a signed token commonly used for stateless authentication.

Competency areas: resource modelling, validation, layered architecture (controller, service, repository), transactions, security, pagination, versioning, caching, observability and automated tests.

Example: in PRJ-002, FRE-003 implemented the proposal and escrow services with JWT authentication and Testcontainers integration tests.

TEC-002 - Beginner, intermediate and advanced competency indicators

Level	Observable indicators
Beginner (L1-L2)	Builds CRUD endpoints; uses Spring Data repositories; limited validation; manual testing.
Intermediate (L3)	Implements validation, global exception handling, pagination and role-based security; writes unit and integration tests; documents the API.
Advanced (L4)	Designs transaction boundaries and idempotency, tunes queries, defines versioning and error contracts, and handles concurrency and failure modes.

TEC-002 - Practical implementation examples

Idempotency: a POST that submits a proposal accepts an idempotency key so repeated clicks do not create duplicates (PRJ-002).

Pagination: list endpoints return pages with a stable sort and a maximum size; keyset pagination is preferred for large tables (PRJ-001).

TEC-002 - Architecture and design considerations

Keep controllers thin; place business rules in services; avoid exposing entities directly, using DTOs (data transfer objects). Define consistent error responses. Use database transactions for multi-step state changes such as ledger entries. Cache only data with clear invalidation rules.

TEC-002 - Testing and maintainability expectations

Expected: JUnit unit tests for business rules; integration tests with a real database (for example Testcontainers PostgreSQL); contract tests or OpenAPI validation; tests for security rules (unauthorized and forbidden cases).

TEC-002 - Common technical mistakes

- Returning HTTP 200 with error text in the body.
- Missing validation and trusting client input.
- N+1 query problems from lazy loading.
- Storing secrets in source code.
- Long transactions that call external services.
- No tests for authorization rules.

TEC-002 - Security, scalability and reliability concerns

Security: follow the OWASP API Security risks; enforce authorization on every endpoint, not only authentication; hash passwords with a modern algorithm; limit rate. Scalability: stateless services, connection pools, caching. Reliability: timeouts, retries with backoff, idempotent writes, health checks.

TEC-002 - Practical questions and assessment scenarios

Scenario or question	What a strong answer shows
How do you prevent duplicate submissions?	Idempotency keys or unique constraints and consistent responses.
How do you secure an endpoint so a freelancer cannot read another freelancer's proposals?	Object-level authorization checks in the service layer plus tests.
A list endpoint is slow at 50,000 rows. What do you check?	Query plan, indexes, pagination, N+1, caching; measures before and after.

TEC-002 - Evidence that demonstrates competency

- OpenAPI specification and service code attributable to the freelancer ID.
- Integration test report.
- Measured before-and-after latency for performance claims.
- Security configuration and authorization tests.

TEC-002 - Distinguishing superficial familiarity from practical experience

Superficial: lists 'Spring Boot' and 'REST', cannot describe transaction boundaries or authorization failures. Practical: describes an incident, the root cause, the fix and the regression test.

A CV line such as 'Spring Boot, 5 years' does not prove advanced proficiency; years of experience can reflect repeated routine work. Evidence of design decisions and tests carries more weight.

TEC-002 - Related skills and alternative terminology

Related skills: Java, Spring Data JPA, Hibernate, Spring Security, OAuth 2.0, JWT, OpenAPI/Swagger, FastAPI (Python alternative), Node.js/Express (alternative), microservices, API gateway, Redis caching.

TEC-002 - Summary of key findings

- API competency is demonstrated by contracts, security rules and tests.
- Years of experience alone do not establish L4.
- Authorization tests are a strong practical signal.

TEC-002 - Keywords and alternative terminology

Spring Boot; REST API; backend; back end; Java; JWT; Spring Security; OpenAPI; Swagger; Testcontainers; idempotency; pagination; microservice; API gateway

TEC-002 - References

- Spring Boot reference documentation (docs.spring.io), OWASP API Security Top 10 (owasp.org), OpenAPI Specification (openapis.org). Named for orientation; links not fetched.

5. TEC-003: TEC-003 Database Design Competency Guide

Source document ID: TEC-003 | Data status: Factual technical guidance (general engineering concepts) with synthetic example references

Purpose and scope: define relational database design competency with PostgreSQL examples and show how to separate real database skill from a listed keyword.

TEC-003 executive summary

Database competency includes schema modelling, constraints, indexing, query analysis, migrations and transaction design. PostgreSQL is used here as the reference relational database; the same principles apply to other SQL systems.

TEC-003 - Technology and competency definitions

PostgreSQL is an open-source relational database management system (RDBMS). A schema is the set of tables, relations and constraints. An index is a data structure that speeds up lookups at the cost of write overhead. A migration is a versioned change to the schema, for example with Flyway or Liquibase. ACID (atomicity, consistency, isolation, durability) describes transaction guarantees.

Example: in PRJ-001, FRE-003 replaced a sequential scan with a composite index on customer and created date; the illustrative latency changed from 1.8 s to 0.4 s. In PRJ-002 the escrow ledger used double-entry tables.

TEC-003 - Beginner, intermediate and advanced competency indicators

Level	Observable indicators
Beginner (L1-L2)	Writes SELECT, INSERT and JOIN queries; creates tables with primary keys; unaware of query plans.
Intermediate (L3)	Normalizes schemas; uses foreign keys and constraints; reads EXPLAIN output; adds appropriate indexes; writes migrations.
Advanced (L4)	Designs for concurrency and consistency, partitions or archives large tables, tunes queries from measured plans, plans zero-downtime migrations.

TEC-003 - Practical implementation examples

Use EXPLAIN (ANALYZE) to compare plans before and after adding an index. A composite index (customer_id, created_at DESC) supports 'latest orders of a customer'.

Ledger: each business event creates balanced debit and credit rows inside one transaction; a reconciliation query checks that totals match.

TEC-003 - Architecture and design considerations

Model entities and relations first; add constraints (NOT NULL, UNIQUE, CHECK, foreign keys) so that invalid states cannot be stored. Use surrogate keys for entities and natural unique constraints for business rules. Keep authoritative values such as proposal price and deadline in relational tables rather than documents. Denormalize only with a measured reason.

TEC-003 - Testing and maintainability expectations

Expected: migration scripts under version control, repeatable seed data, integration tests against a real PostgreSQL instance, documented indexes and their purpose.

TEC-003 - Common technical mistakes

- Indexing every column.
- Storing multiple values in one column.
- Missing foreign keys.
- Running schema changes manually in production.
- Using floating-point types for money instead of numeric/decimal.
- Ignoring transaction isolation for concurrent updates.

TEC-003 - Security, scalability and reliability concerns

Security: least-privilege database roles, parameterized queries against SQL injection, encryption at rest and backups. Scalability: indexes, partitioning, read replicas. Reliability: backups with tested restores, constraints and migrations rehearsed on a copy.

TEC-003 - Practical questions and assessment scenarios

Scenario or question	What a strong answer shows
A query slows when the table grows. What do you do?	Measure with EXPLAIN, identify scan type, adjust indexes or query, verify improvement.
How do you store money?	Numeric/decimal with fixed scale, currency code, never floating point.
How do you change a column used by a live service?	Expand-and-contract migration in several releases.

TEC-003 - Evidence that demonstrates competency

- Schema and migration files attributable to the freelancer ID.
- Query plans before and after.
- Integration tests with a real database.
- Documentation of index rationale.

TEC-003 - Distinguishing superficial familiarity from practical experience

Superficial: lists PostgreSQL, uses it only through an ORM defaults, cannot explain an index. Practical: shows a plan, a measured improvement and the trade-off in write cost.

'PostgreSQL' on a CV is not proof of advanced proficiency; many projects use the database without tuning or design decisions. FRE-003 has documented tuning evidence in PRJ-001; FRE-001 has documented schema design evidence in PRJ-001 and PRJ-002.

TEC-003 - Related skills and alternative terminology

Related skills: SQL, MySQL (related relational system), Flyway, Liquibase, JPA/Hibernate, ChromaDB (vector store, different category), Redis (cache), data modelling, normalization.

TEC-003 - Summary of key findings

- Database skill is shown by schemas, plans and measured improvements.
- Authoritative proposal numbers belong in relational tables.
- Money must not use floating-point types.

TEC-003 - Keywords and alternative terminology

PostgreSQL; Postgres; SQL; relational database; index; EXPLAIN; migration; Flyway; schema design; ACID; transaction; normalization; query tuning

TEC-003 - References

- PostgreSQL documentation (postgresql.org/docs), Flyway documentation (documentation.red-gate.com). Named for orientation; links not fetched.

6. TEC-004: TEC-004 AI and Machine Learning Engineering Competency Guide

Source document ID: TEC-004 | Data status: Factual technical guidance (general engineering concepts) with synthetic example references

Purpose and scope: define competency for applied machine learning and retrieval-augmented generation (RAG) and the evidence needed to trust a freelancer's AI claims.

TEC-004 executive summary

AI/ML competency requires data preparation, modelling, evaluation, deployment and monitoring. For RAG systems it also requires chunking, embedding, hybrid retrieval, citation and retrieval evaluation. Model accuracy claims without evaluation protocol are weak evidence.

TEC-004 - Technology and competency definitions

Machine learning (ML) learns patterns from data. scikit-learn is a Python library for classical ML. PyTorch is a deep-learning framework. An embedding is a numeric vector representing text so similar meaning is close in vector space. RAG (retrieval-augmented generation) retrieves relevant passages and gives them to a language model so answers can cite sources. Hybrid retrieval combines semantic vector search with keyword search such as BM25. ChromaDB is a vector database. Recall@K measures whether a relevant document appears in the top K results; MRR (mean reciprocal rank) and nDCG@K measure ranking quality.

Example: in PRJ-004, FRE-004 built ingestion, chunking, hybrid retrieval and citations and measured Recall@5 of 0.88 on 60 questions (illustrative).

TEC-004 - Beginner, intermediate and advanced competency indicators

Level	Observable indicators
Beginner (L1-L2)	Runs notebooks and tutorials; trains a model with defaults; evaluates on the training data.
Intermediate (L3)	Builds train/test splits, baselines and metrics; serves a model through an API; builds a basic RAG pipeline with metadata filters and citations.
Advanced (L4)	Designs evaluation protocols, handles drift and cold start, monitors in production, compares retrieval strategies with measured metrics, and documents failure modes.

TEC-004 - Practical implementation examples

Time-based split: train on older purchases and test on newer ones to avoid leaking future information (PRJ-003).

Section-aware chunking: split a document by headings, repeat the table caption in each chunk, and store metadata (document_id, category, entity IDs) so retrieval can filter (PRJ-004).

TEC-004 - Architecture and design considerations

Always compare against a simple baseline (popularity, keyword search). Keep the numerical scoring deterministic and outside the language model; use the model to explain. Version the embedding model and re-embed when it changes. Store provenance with each chunk.

TEC-004 - Testing and maintainability expectations

Expected: reproducible training scripts, fixed random seeds, evaluation notebooks or reports, retrieval test sets with expected document IDs, regression checks when chunking or models change.

TEC-004 - Common technical mistakes

- Evaluating on training data.
- Data leakage across splits.
- No baseline.
- Chunks that split tables or lists and lose meaning.
- Letting the language model calculate scores or invent citations.
- A tiny evaluation set presented as proof of accuracy.

TEC-004 - Security, scalability and reliability concerns

Security and privacy: avoid personal data in prompts and indexes; control access by metadata filters. Scalability: batch embedding, index size, latency. Reliability: monitoring retrieval quality, fallbacks when no evidence is found, refusing to answer without sources.

TEC-004 - Practical questions and assessment scenarios

Scenario or question	What a strong answer shows
How do you know your retriever works?	Test questions with expected document IDs; Recall@K, MRR, nDCG; error analysis.
How do you handle cold-start items in a recommender?	Content-based fallback, popularity priors, explicit limitations.
Why not let the LLM compute the score?	Non-determinism and arithmetic errors; scoring must be reproducible.

TEC-004 - Evidence that demonstrates competency

- Evaluation reports with metric definitions.
- Test set with expected sources.
- Training and ingestion code attributable to the freelancer ID.
- Deployment and monitoring evidence for production claims.

TEC-004 - Distinguishing superficial familiarity from practical experience

Superficial: mentions 'AI' and 'LLM', cannot describe evaluation or failure cases. Practical: shows metrics, a baseline, and a documented limitation (for example PRJ-003 cold start).

A CV listing Python and machine learning does not prove production competence: notebook experiments differ from monitored systems. FRE-004 has documented RAG and recommender evidence but none for production monitoring.

TEC-004 - Related skills and alternative terminology

Related skills: Python, NumPy, pandas, scikit-learn, PyTorch, TensorFlow, embeddings, ChromaDB, FAISS, BM25, LangChain-style orchestration, prompt engineering, MLOps, model monitoring.

TEC-004 - Summary of key findings

- AI competency is evidenced by evaluation, baselines and documented limits.
- Retrieval quality must be measured with Recall@K, MRR or nDCG@K.
- Scores must be computed by deterministic code, not by the LLM.

TEC-004 - Keywords and alternative terminology

machine learning; ML; AI; RAG; retrieval-augmented generation; embeddings; ChromaDB; vector database; hybrid retrieval; BM25; Recall@K; MRR; nDCG; Python; scikit-learn; PyTorch; FastAPI; recommender

TEC-004 - References

- scikit-learn user guide (scikit-learn.org), ChromaDB documentation (docs.trychroma.com), PyTorch documentation (pytorch.org). Named for orientation; links not fetched.

7. TEC-005: TEC-005 DevOps and Cloud Engineering Competency Guide

Source document ID: TEC-005 | Data status: Factual technical guidance (general engineering concepts) with synthetic example references

Purpose and scope: define DevOps and cloud competency and describe evidence that distinguishes real operational experience from a list of tools.

TEC-005 executive summary

DevOps competency means shipping software repeatably and operating it reliably. Docker, Kubernetes, CI/CD, Terraform and monitoring are common tools, but staging success is not the same as production operation.

TEC-005 - Technology and competency definitions

Docker packages an application and its dependencies in a container image. Kubernetes (K8s) orchestrates containers across machines with deployments, services and health probes. CI/CD (continuous integration and continuous delivery) automates build, test and deployment. Terraform is an infrastructure-as-code (IaC) tool that describes cloud resources declaratively. Observability uses metrics, logs and traces; Prometheus and Grafana are common metrics and dashboard tools.

Example: in PRJ-005, FRE-005 provisioned AWS with Terraform and migrated services to a staging Kubernetes cluster, while production cut-over remained incomplete.

TEC-005 - Beginner, intermediate and advanced competency indicators

Level	Observable indicators
Beginner (L1-L2)	Writes a Dockerfile; runs containers locally; follows an existing pipeline.
Intermediate (L3)	Builds multi-stage images; authors CI pipelines with test stages; deploys to Kubernetes with probes; writes Terraform modules; sets alerts.
Advanced (L4)	Designs release strategies (blue-green, canary), secures supply chains, rehearses rollback and disaster recovery, controls cost, and writes runbooks.

TEC-005 - Practical implementation examples

Multi-stage Docker build: compile in a builder image, copy only the artifact into a small runtime image, run as a non-root user.

Pipeline: lint, unit tests, build image, scan image, push, deploy to staging, smoke test, manual approval, deploy to production. Layer caching cut one pipeline from over 25 to about 11 minutes in PRJ-005 (illustrative).

TEC-005 - Architecture and design considerations

Keep environments reproducible from code. Separate staging and production accounts. Use health and readiness probes. Store secrets in a secret manager, not in images. Treat stateful services carefully: they may not tolerate pod restarts (PRJ-005).

TEC-005 - Testing and maintainability expectations

Expected: pipeline-as-code in version control, smoke tests after deployment, rollback procedure, documented runbooks, alert rules tested at least once.

TEC-005 - Common technical mistakes

- Running containers as root.
- Secrets baked into images.
- No resource limits.
- Manual production changes outside IaC.
- Dashboards without alert thresholds.
- Declaring a migration complete after staging only.

TEC-005 - Security, scalability and reliability concerns

Security: image scanning, least-privilege IAM, network segmentation. Scalability: horizontal scaling, autoscaling policies. Reliability: health checks, backups with tested restores, failure drills.

TEC-005 - Practical questions and assessment scenarios

Scenario or question	What a strong answer shows
A stateful service fails after a pod restart. What now?	Identifies state handling, uses StatefulSet or keeps it outside the cluster, documents risk.
How do you roll back a bad release?	Versioned images, automated rollback, database migration compatibility.
How do you prove a pipeline is trustworthy?	Test gates, reproducible builds, audit trail.

TEC-005 - Evidence that demonstrates competency

- Pipeline definitions and run history.
- Terraform code and plan outputs.
- Runbooks and rollback rehearsal notes.
- Dashboards and alert rules.
- Production deployment logs for production claims.

TEC-005 - Distinguishing superficial familiarity from practical experience

Superficial: lists Kubernetes after running a tutorial cluster. Practical: describes a failed deployment, how it was detected and how it was recovered.

A CV listing Docker and Kubernetes does not prove production operations. FRE-005 has staging Kubernetes evidence in PRJ-005 but not completed production operation.

TEC-005 - Related skills and alternative terminology

Related skills: Docker Compose, Helm, GitHub Actions, GitLab CI, Jenkins, Terraform, Ansible, AWS, Azure, GCP, Prometheus, Grafana, site reliability engineering (SRE), DevSecOps.

TEC-005 - Summary of key findings

- DevOps competency requires evidence of operation, not only provisioning.
- Staging-only evidence must be labelled as staging-only.
- Rollback rehearsal and runbooks are strong signals.

TEC-005 - Keywords and alternative terminology

DevOps; Docker; Kubernetes; K8s; CI/CD; GitHub Actions; Terraform; infrastructure as code; AWS; Prometheus; Grafana; monitoring; container; pipeline; cloud engineer

TEC-005 - References

- Docker documentation (docs.docker.com), Kubernetes documentation (kubernetes.io/docs), Terraform documentation (developer.hashicorp.com/terraform). Named for orientation; links not fetched.

8. TEC-006: TEC-006 Mobile Engineering Competency Guide

Source document ID: TEC-006 | Data status: Factual technical guidance (general engineering concepts) with synthetic example references

Purpose and scope: define mobile engineering competency with Flutter and Dart examples, including offline behaviour, notifications and release readiness.

TEC-006 executive summary

Mobile competency covers UI composition, state management, local persistence, networking, notifications, performance on varied devices and store release. A working demo build is weaker evidence than a tested release on each platform.

TEC-006 - Technology and competency definitions

Flutter is a cross-platform UI toolkit that compiles Dart code to Android, iOS and other targets. Dart is the programming language used by Flutter. A widget is a UI building block. Offline-first design lets an app read and write local data when the network is unavailable and synchronize later. Firebase Cloud Messaging (FCM) delivers push notifications.

Example: in PRJ-006, FRE-006 built itinerary, map and booking screens, SQLite storage and FCM notifications, and published an Android internal-testing build. iOS release evidence is absent.

TEC-006 - Beginner, intermediate and advanced competency indicators

Level	Observable indicators
Beginner (L1-L2)	Builds screens from a design; fetches data from a REST API; limited tests.
Intermediate (L3)	Uses structured state management (Provider, Riverpod, Bloc); handles loading and error states; adds local storage; writes widget tests; handles permissions.
Advanced (L4)	Designs conflict-resolving sync, profiles performance on low-end devices, manages signing and store releases for both platforms, and writes platform channels where needed.

TEC-006 - Practical implementation examples

Last-write-wins sync with a visible conflict banner is a simple strategy; it can lose edits, so field-level merge or server-side versioning is needed for strong guarantees (PRJ-006 stopped at the simple strategy).

Memoizing widgets and using const constructors reduces unnecessary rebuilds on map screens.

TEC-006 - Architecture and design considerations

Separate UI, domain and data layers. Define repository interfaces so offline cache and remote API can be swapped. Keep API contracts in OpenAPI. Plan for platform differences in permissions and notifications.

TEC-006 - Testing and maintainability expectations

Expected: unit tests for domain logic, widget tests for key screens, integration tests on devices or emulators, and device-farm runs for release candidates. Coverage figures should be documented.

TEC-006 - Common technical mistakes

- Business logic inside widgets.
- No offline conflict strategy.
- Ignoring older devices.
- Testing only on one emulator.
- Claiming iOS support without an iOS build.
- Hard-coded API keys.

TEC-006 - Security, scalability and reliability concerns

Security: secure storage for tokens, certificate handling, no secrets in the app bundle. Scalability: paginated API calls and image caching. Reliability: crash reporting, retry policies, graceful degradation offline.

TEC-006 - Practical questions and assessment scenarios

Scenario or question	What a strong answer shows
How do you handle edits made offline?	Queue, conflict strategy, user feedback, tests for conflicts.
How do you prove iOS readiness?	Signed iOS build, TestFlight or equivalent record, platform-specific tests.
How do you find slow screens?	Profile mode, timeline, rebuild counts, low-end device runs.

TEC-006 - Evidence that demonstrates competency

- Build artifacts per platform.
- Widget and integration tests.
- Crash and performance reports.
- Store or beta distribution records.

TEC-006 - Distinguishing superficial familiarity from practical experience

Superficial: lists Flutter after one demo app. Practical: describes platform-specific problems and how they were tested.

A CV listing Flutter does not prove release competence on both stores. FRE-006 has documented Android internal-testing evidence only.

TEC-006 - Related skills and alternative terminology

Related skills: Dart, Flutter, Android (Kotlin), iOS (Swift), React Native (alternative), SQLite, Drift, Firebase, Riverpod, Bloc, mobile CI/CD, app store release.

TEC-006 - Summary of key findings

- Mobile competency requires per-platform evidence.
- Offline conflict handling is a distinguishing indicator.
- Absence of iOS evidence is missing information, not proof of inability.

TEC-006 - Keywords and alternative terminology

Flutter; Dart; mobile app; Android; iOS; offline-first; SQLite; Firebase Cloud Messaging; push notifications; widget test; cross-platform; mobile developer

TEC-006 - References

- Flutter documentation (flutter.dev), Dart documentation (dart.dev), Firebase documentation (firebase.google.com/docs). Named for orientation; links not fetched.

9. QUA-001: QUA-001 Code Quality and Maintainability Indicators

Source document ID: QUA-001 | Data status: Factual guidance on quality indicators with synthetic (fictional) examples clearly labelled

Purpose and scope: define objective, observable indicators of code quality and maintainability that can be supported by evidence when evaluating freelancer proposals.

QUA-001 executive summary

Code quality is judged by observable indicators: readable structure, consistent style, tests, low duplication, documented interfaces and manageable complexity. Opinions such as 'clean code' without evidence are weak.

QUA-001 - Observable indicators

Indicator	How it is observed	Evidence type
Readable structure	Layered architecture, small functions, naming conventions	Code review notes
Static analysis results	Linter and analyzer warnings per 1,000 lines	Tool report
Test coverage and test quality	Line or branch coverage plus meaningful assertions	Coverage report
Duplication	Percentage of duplicated blocks	Static analysis
API documentation	OpenAPI or typed contracts	Specification file
Type safety	Strict TypeScript, few uses of any	Compiler configuration
Review responsiveness	Number of review comments resolved before merge	Pull request history

QUA-001 - Coverage is not quality

A 90 percent coverage figure with assertions that test nothing is weak evidence. Prefer evidence of tests that guard past defects, for example the idempotency test added to PRJ-002 after duplicate proposals appeared (FRE-003) or the filtering component tests in PRJ-001 (FRE-002).

QUA-001 - Maintainability expectations

- New developers can run the project from the README.
- Configuration is externalized.
- Database changes use versioned migrations.
- Dependencies are pinned and updated with a recorded process.
- Architecture decisions are documented.

QUA-001 - Facts versus opinions

Type	Definition	Example (fictional)
Documented fact	Observable, verifiable record	Milestone 3 delivered 2 days after the agreed date (record DEL-007).
Measured indicator	Quantified from a record or tool	Illustrative line coverage 71 percent in PRJ-001.
Subjective opinion	Personal impression without a checkable basis	'Felt slow' (REV-018).

Subjective statements about code quality must be labelled as opinions in recommendations and must not be turned into scores without supporting records.

QUA-001 - Examples (fictional)

- Positive: FRE-003 documented the PRJ-001 index change, the before and after query plan, and added a regression test.
- Negative: A fictional repository has no README, two copies of the same pricing function and no tests; maintainability risk is high.
- Mixed: PRJ-001 backend coverage was 71 percent but frontend coverage was not measured; the correct statement is 'frontend coverage unknown'.

QUA-001 - Evidence requirements and uncertainty

Quality claims require at least one artifact: code sample, report or review record. When no artifact exists, the system reports 'no code quality evidence found' and does not infer low quality.

QUA-001 - Summary of key findings

- Use observable indicators with artifacts.
- Coverage numbers need context.
- Missing evidence is not poor quality.

QUA-001 - Keywords and alternative terminology

code quality; maintainability; static analysis; linting; coverage; technical debt; clean code; code review; duplication

10. QUA-002: QUA-002 Testing, Defect Handling and Reliability Indicators

Source document ID: QUA-002 | Data status: Factual guidance on quality indicators with synthetic (fictional) examples clearly labelled

Purpose and scope: define how testing practice and defect handling are evidenced, and how to interpret open defects in project history.

QUA-002 executive summary

Reliable delivery shows a test strategy, defect records and regression protection. Open defects at closure are common; what matters is whether they are disclosed, understood and tracked.

QUA-002 - Test types and evidence

Test type	Purpose	Evidence
Unit	Verify one component	Test files, report
Integration	Verify components with real dependencies	Testcontainers run, API test report
End-to-end	Verify user flows	Cypress or Playwright run
Retrieval evaluation (RAG)	Verify expected documents are retrieved	Question set with expected IDs, Recall@K
Load	Verify capacity	Load test report

QUA-002 - Defect handling indicators

- Defects recorded with reproduction steps.
- Severity and status tracked.
- Fixes accompanied by regression tests.
- Open defects disclosed at handover with a risk statement.

QUA-002 - Interpreting open defects

In the fictional PRJ-002, an escrow reconciliation mismatch of 0.4 percent of test transactions stayed open at closure. FRE-003 disclosed it and the feature shipped behind a flag. A disclosed and contained defect is a limitation, not proof of incompetence. A hidden defect would be a larger concern.

QUA-002 - Reliability indicators

- Post-release defect counts per delivered component (delivery records DEL-001 to DEL-015, illustrative).
- Rollback or flag mechanisms.
- Monitoring and alert rules.
- Retries and idempotency in integrations.

QUA-002 - Example (fictional)

PRJ-004: retrieval evaluated on 60 questions; Recall@5 was 0.88 (illustrative). Two questions still cited a superseded policy version; this is documented and tracked. The evaluation set is small, so the figure should be reported with that caution.

QUA-002 - Evidence requirements and uncertainty

Claims such as 'well tested' need a test artifact. If only a claim exists, label it unsupported. Do not treat missing test records as zero quality, and do not treat a single defect as a general reliability finding.

QUA-002 - Summary of key findings

- Testing evidence consists of artifacts and results.
- Disclosed open defects are limitations.
- Small test sets limit conclusions.

QUA-002 - Keywords and alternative terminology

testing; QA; defect; bug; regression; reliability; Testcontainers; integration tests; Recall@K; post-release defects

11. QUA-003: QUA-003 Communication and Collaboration Indicators

Source document ID: QUA-003 | Data status: Factual guidance on quality indicators with synthetic (fictional) examples clearly labelled

Purpose and scope: define observable indicators of communication and collaboration that can be recorded without relying on personality judgements.

QUA-003 executive summary

Communication quality is observable through response times, clarity of requirement questions, status reports and handover documents. Style preferences and personality are not evidence.

QUA-003 - Observable indicators

Indicator	Observation	Illustrative example
Response time	Median time to first reply on working days	FRE-001: about 6 working hours (illustrative)
Requirement clarification	Questions raised before building	FRE-002 asked clarifying questions on 9 of 11 tickets (illustrative)
Status reporting	Frequency and content of reports	FRE-001 weekly written status in PRJ-001 and PRJ-002
Demo cadence	Working builds shown at intervals	FRE-006 delivered demo builds every two weeks
Handover documentation	Runbooks, READMEs	FRE-005 runbook-style handovers

QUA-003 - Collaboration in team projects

When several freelancers work on one project, observable signs include documented interface agreements, shared pull request reviews and clear ownership. Collaboration evidence for PRJ-001 includes the API contract between FRE-002 and FRE-003.

QUA-003 - Facts versus opinions

Type	Definition	Example (fictional)
Documented fact	Observable, verifiable record	Milestone 3 delivered 2 days after the agreed date (record DEL-007).
Measured indicator	Quantified from a record or tool	Illustrative line coverage 71 percent in PRJ-001.
Subjective opinion	Personal impression without a checkable basis	'Felt slow' (REV-018).

A comment such as 'communication style was not my favourite' (REV-006) is subjective and does not indicate poor communication practice.

QUA-003 - Time zones, language and availability

Differences in time zone or language are not quality indicators. Compare stated overlap hours with the client's needs, and treat overlap as a scheduling fact.

QUA-003 - Evidence requirements and uncertainty

Communication indicators need records such as message logs, reports or review comments with specifics. Missing records mean 'not evidenced'. They must not lower a score as if the freelancer communicated poorly.

QUA-003 - Summary of key findings

- Use response time, clarification and reporting as observable indicators.
- Style preference is not evidence.
- Missing records are not negative.

QUA-003 - Keywords and alternative terminology

communication; collaboration; responsiveness; status report; requirement clarification; handover; teamwork

12. Summary of key findings

- TEC-001: Frontend competency is shown by features, tests and decisions, not by technology lists.
- TEC-002: API competency is demonstrated by contracts, security rules and tests.
- TEC-003: Database skill is shown by schemas, plans and measured improvements.
- TEC-004: AI competency is evidenced by evaluation, baselines and documented limits.
- TEC-005: DevOps competency requires evidence of operation, not only provisioning.
- TEC-006: Mobile competency requires per-platform evidence.
- QUA-001: Use observable indicators with artifacts.
- QUA-002: Testing evidence consists of artifacts and results.
- QUA-003: Use response time, clarification and reporting as observable indicators.

13. Keywords and alternative terminology

Angular; frontend engineering; SPA; TypeScript; RxJS; NgRx; state management; Jasmine; Karma; accessibility; WCAG; lazy loading; front end; UI developer; web UI; Spring Boot; REST API; backend; back end; Java; JWT; Spring Security; OpenAPI; Swagger; Testcontainers; idempotency; pagination; microservice; API gateway; PostgreSQL; Postgres; SQL; relational database; index; EXPLAIN; migration; Flyway; schema design; ACID; transaction; normalization; query tuning; machine learning; ML; AI; RAG; retrieval-augmented generation; embeddings; ChromaDB; vector database; hybrid retrieval; BM25; Recall@K; MRR; nDCG; Python; scikit-learn; PyTorch; FastAPI; recommender; DevOps; Docker; Kubernetes; K8s; CI/CD; GitHub Actions; Terraform; infrastructure as code; AWS; Prometheus; Grafana; monitoring; container; pipeline; cloud engineer; Flutter; Dart; mobile app; Android; iOS; offline-first; SQLite; Firebase Cloud Messaging; push notifications; widget test; cross-platform; mobile developer; code quality; maintainability; static analysis; linting; coverage; technical debt; clean code; code review; duplication; testing; QA; defect; bug; regression; reliability; integration tests; post-release defects; communication; collaboration; responsiveness; status report; requirement clarification; handover; teamwork